Infraestrutura de software

Guia de Relatorios

Aluno: Vinicius de Souza Barbosa | Login: vsb
Atividade: Implementação 4 — Sincronização do Metrô | Data: 04/11/2026
Repositório: https://github.com/venici-o/Syncronization_VSB

Atenção

- **Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.**
- **O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.**
- **Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.**

1. Objetivo

Implementar em C a sincronização do embarque em carros de metrô controlados por threads. Garantir que cada passageiro reserve uma vaga disponível, que os embarques possam ocorrer simultaneamente e que o carro aguarde as confirmações antes de partir. Entregar o código, Makefile, README, testes reproduzíveis, evidências e este relatório.

2. Checklist de Requisitos

Requisito do enunciado	Status	Como testar / evidência
Código C executável em Linux	OK	make; resultados/validacao.log registra CachyOS Linux, GCC 16.2.1 e GNU Make 4.4.1.
struct station e quatro funções exigidas	OK	sync_car.h e sync_car.c; make check.
Mutex e condições pthread exigidos	OK	Uso de init, lock, unlock, wait, signal e broadcast na implementação.
Um mutex por station; sem semáforos	OK	Inspeção de sync_car.h e sync_car.c.
Reserva dentro da capacidade	OK	T03, T05 e T06 em resultados/validacao.log.
Embarque simultâneo	OK	T07: três reservas antes de qualquer confirmação.

Carro espera todos os embarques	OK	T04–T07: com última confirmação retida, carro_retornou=0.
Carro parte cheio ou sem mais passageiros esperando	OK	T01–T06; vagas são zeradas antes da saída.
Ausência de busy-wait na estação	OK	Os laços de espera em sync_car.c usam pthread_cond_wait.
Makefile gera sync_car e oferece limpeza	OK	make clean; make; execução de ./sync_car.
README e relatório preenchido	OK	README.md e vsb.pdf completos.
Publicação e envio no formato exigido	Não feito	Publicar os arquivos novos no GitHub; preparar vsb.tar e vsb.pdf e enviar ao Classroom.

3. Como Reproduzir

Comandos reais, copiáveis. Se não rodar com o que está aqui, não é avaliado.

Ambiente validado: CachyOS Linux x86_64, kernel 7.2.6-1-cachyos, GCC 16.2.1 e GNU Make 4.4.1. Executar os comandos na pasta que contém o Makefile.

Compilar:

```
make clean
make
```

Executar (exemplo):

```
./sync_car
```

Verificação de sintaxe: make check
Suíte completa: make test
Cenário específico, após make test: ./sync_car-tests --cenario 5 2 3

O runner original termina com “Looks good!”. A suíte adicional termina com “[OK FINAL] runner original + 7 cenarios controlados + 4 entradas invalidas”. O cenário específico significa cinco passageiros e carros sucessivos com duas e três vagas.

Para registrar outra validação completa:
mkdir -p resultados
script -q -e -c 'sh testes/validar.sh' resultados/nova_validacao.log

Os testes controlados exigem suporte a --wrap no linker (GNU ld no ambiente validado). O runner original mantém o aviso preexistente sobre o parâmetro foo não utilizado. O alvo make clean remove os executáveis e objetos, preservando fontes e evidências.

4. Arquitetura

Arquivos e responsabilidades:

1. **sync_car.c → Inicialização da estação, reserva, confirmação de embarque e controle da partida.**
2. **sync_car.h → Definição da struct station e declarações das quatro funções.**

3. sync_car-runner.c → Runner original, com 100 passageiros e capacidades aleatórias.

4. testes/sync_car-tests.c → Sete cenários controlados, observação das esperas e validação da entrada de testes.

5. testes/executar.sh → Executa o runner, os sete cenários e quatro entradas inválidas.

6. testes/validar.sh → Recompila do zero e registra ambiente, testes e hashes.

7. Makefile e README.md → Compilação, limpeza, testes e instruções de reprodução.

Decisões técnicas:

1. **Usar um mutex por estação → Proteger consulta e atualização dos contadores juntas → Evitar reservas concorrentes da mesma vaga.**
2. **Separar reserva e confirmação → Permitir vários passageiros embarcando ao mesmo tempo → O contador p_embarcando impede a partida antecipada.**
3. **Usar duas condições e esperar em while → Reavaliar o estado após cada aviso → Passageiros aguardam em carro_disp e o carro aguarda em carro_pronto.**

O carro permanece enquanto há passageiros embarcando OU enquanto há vagas E passageiros aguardando. Antes de partir, zera as vagas. A implementação pressupõe um carro por vez, capacidade não negativa e uma confirmação por reserva; não garante ordem FIFO.

5. Estratégias e Diário de Desenvolvimento

5.1 Estratégias da semana

Foi utilizada apenas S1; as linhas S2 e S3 abaixo não se aplicam.

Estratégia	Nome curto	Contexto	Motivo da troca
S1	Contadores + mutex + condições	Implementação incremental de inicialização, reserva, confirmação e carregamento.	Não houve troca de abordagem; foram corrigidas a condição de saída e a retirada das vagas.
S2			
S3			

5.2 Diário de Tentativas

H indica a linha numerada de resultados/historico_legivel.txt, derivada da cópia preservada do log. Os eventos

antigos seguem sua ordem no registro, sem atribuir horários inexistentes a cada comando. V indica resultados/validacao.log; R indica resultados/regressoes.log. Datas e horários usam America/Recife.

1	S1	Compilar station_init	Falha	Faltava ponto e vírgula antes da inicialização do mutex.	04/11; ordem 1	H160
2	S1	Verificar station_load_car	Parcial: avisos	Uso de & na comparação: endereços em vez de valores.	04/11; ordem 2	H239 e H242
3	S1	Executar make run	Falha	Alvo não existia. Foi adotada execução direta.	04/11; ordem 3	H301
4	S1	Executar o runner	Falha: timeout	Revisar condição de partida e vagas residuais; hash do binário daquela tentativa não registrado.	04/11; ordem 4	H336 e H358
5	S1	Recompilar do zero e verificar sintaxe	OK		04/11 19:29:17	V: make clean, make e make check
6	S1	Executar runner e sete cenários controlados	OK		04/11 19:29:17–18	V: Looks good! e T01–T07
7	S1	Testar quatro entradas inválidas	OK: rejeitadas		04/11 19:29:17–18	V: retorno esperado 2
8	S1	Verificar detecção dos defeitos anteriores	OK: detectados		04/11 19:29:48	R: R01 e R02

O Git registra station_load_car em e3fe031 (04/11, 19:01:31) e a correção das vagas em ec9cbd0 (19:12:39); ver resultados/historico_git.txt. São horários de commits, não horários atribuídos aos comandos do log. A dificuldade principal foi distinguir reserva de confirmação e perceber que compilar não garante a correção do embarque.

6. Evidências

6.1 Log automático (faz o trabalho pesado

T01 — 0 passageiros e 0 vagas: retorno com os três contadores zerados. Resultado: aprovado.

T02 — 0 passageiros e 10 vagas: carro saiu vazio e retirou as vagas restantes. Resultado: aprovado.

T03 — 3 passageiros; carros com 0 e 3 vagas: o primeiro deixou todos esperando; o segundo atendeu os três. Resultado: aprovado.

T04 — 2 passageiros e 5 vagas: duas reservas, espera pelas duas confirmações e retirada das vagas restantes. Resultado: aprovado.

T05 — 4 passageiros e 4 vagas: quatro reservas antes das confirmações, sem exceder a capacidade. Resultado: aprovado.

T06 — 5 passageiros; carros com 2 e 3 vagas: dois embarcaram no primeiro e três no segundo. Resultado: aprovado.

T07 — 3 passageiros; última confirmação retida: houve três reservas antes de qualquer confirmação; com p_embarcando=1, o carro ainda não retornou. Após a última confirmação, retornou. Resultado: aprovado.

Entradas inválidas da interface de testes: --cenario -1 2; --cenario 2 -1; --cenario abc 2; --cenario 2 1. Todas foram rejeitadas com retorno 2 antes de criar threads. O último caso não oferece carros suficientes para concluir o teste.

Esses casos inválidos validam o executável adicional, não a API original. A API pressupõe capacidade não negativa; o enunciado não define tratamento de capacidade negativa. A implementação do aluno não foi alterada para inserir essa validação.

Método: o linker observa pthread_cond_wait para identificar chegadas e novas esperas do carro. Um controlador segura as confirmações até as reservas terminarem e libera uma por vez. Os estados são lidos sob mutex; o controlador também espera com mutex e condição. Não usa sleep ou polling para definir a ordem. O alarme de dez segundos apenas detecta bloqueios. O mutex adicional pertence ao controlador do teste, não à struct station.

Verificação dos testes: uma cópia temporária da versão histórica e3fe031 falhou em T02 por deixar dez vagas residuais. Outra cópia, com o teste de p_embarcando retirado deliberadamente, falhou por partir com confirmação pendente. Essas execuções estão identificadas em regressoes.log; a implementação atual foi preservada.

6.2 Prints

• Erro principal: o momento do bug mais relevante da semana.



Captura atual do visualizador de resultados/regressoes.log (R01): reexecução da versão histórica e3fe031. O carro retornou deixando dez vagas. Não é um print antigo do momento da falha original do aluno.

• Validação final: teste passando / saída correta.



Captura do visualizador de resultados/validacao.log: T07 registra três reservas antes das confirmações e a espera pela última confirmação. O runner original e os sete cenários controlados passaram; quatro entradas inválidas também foram rejeitadas.

Prints extras só se algo excepcional aconteceu e o log sozinho não explica, você precisa ter esse feeling, mas posso dar uma ajuda (ex.: comportamento visual, crash com stack trace longo).

7. Uso de IA

Onde usei IA:

Usei IA para entender mutex, condições e as funções da estação, escrevendo o código e pedindo revisões. Depois solicitei explicitamente que a IA criasse os testes e preenchesse o relatório. A IA recomendou os testes adicionais, fez scripts, ajustes no Makefile, README e relatório, além de embelezar a validação automatizada.

Prompts principais (liste, sem história):

• “quero que você me explique de maneira didática o que precisa ser feito e me guie, de forma compreensiva, para que EU faça o necessário da implementação. faça uma explicação, por completo, da atividade descrita”

• “me explica mais tecnicamente como funciona o mutex na struct”

• “tenta evitar me falar o codigo direto só me explica como funciona”

• “tu consegue rodar os casos de exceção e printar pra mim de maneira que encaixe no relatorio???”

• “terminei o relatório se puder checar pra mim e me sinalizar os erros seria bom”

O que validei manualmente e como:

Construí, executei checagens de sintaxe, compilei e executei o runner durante o desenvolvimento; meu log registra tentativas, avisos e falhas. Informei na conversa que obtive “Looks good!”. A aprovação reproduzível da seção 6 vem da execução automatizada registrada em validacao.log. Os testes adicionais foram executados pela IA a meu pedido; não são apresentados como validação manual minha. A leitura final do relatório e o envio ainda dependem de mim.

8. Reflexão Final

Reflexão:

Comecei com mais conhecimento teórico do que prático sobre threads. Durante a implementação, entendi a diferença entre reservar uma vaga e confirmar o embarque, além de como o mutex protege os contadores. Também compreendi como as condições permitem esperar sem impedir que outras threads avancem. Os testes mostraram que compilar não garante o funcionamento correto, por isso, definiria os estados e cenários de teste desde o início.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	Sim, na validação automatizada: make clean, make e make check em validacao.log.
Rodei os testes e evidencias.log foi gerado	Sim. Histórico e validação em resultados/evidencias_completas.log; usar como evidencias.log na entrega.
Tenho prints obrigatórios	Capturas do erro histórico reproduzido e da validação final incluídas. Não há captura antiga original do erro.
Testei pelo menos um caso limite e um caso inválido	Sim: T01–T07 e quatro entradas inválidas da interface de testes.
Preenchi a seção de uso de IA	Sim, distinguindo código do aluno e trabalho automatizado.
Revisei o relatório e removi frases genéricas / vazias	Sim, escrita e revisão técnica manual; leitura final e checagem de erros por inteligência artificial.

10. Se eu tivesse mais 2 horas

Com mais tempo, testaria passageiros chegando durante o embarque e duas estações funcionando de

forma independente. Também melhoraria o tratamento de erros e definiria como a API deve lidar com parâmetros inválidos. Revisaria o runner para substituir a espera contínua e o controle com volatile, além de avaliar a cobertura dos testes e possíveis condições de corrida. Por fim, testaria outras plataformas e uma alternativa ao --wrap.